

# 03 - Frontend Architecture Audit

Olusturulma: 14 Nisan 2026 19:49 Kaynak: docs/audit Format: Tekil rapor

## 03 - Frontend Architecture Audit

Tarih: 2026-04-14 Kapsam: frontend state kaynakları, route yapısı, ekranlar arası veri akışı, sipariş/masa/ ödeme/ayarlar ekranları, component sorumlulukları, tekrar eden mantıklar ve kırılğan koşullar. Sınır: Bu raporda uygulama kodu değiştirilmedi; yalnızca analiz, state akışı çıkarımı ve raporlama yapıldı.

Bağlam olarak dikkate alınan raporlar:

- docs/audit/00-overall-audit.md
- docs/audit/00a-prioritized-action-plan.md
- docs/audit/00b-first-hardening-pass.md
- docs/audit/01-product-business-rules.md
- docs/audit/02-ux-ui-audit.md

Not: Bu denetim mevcut çalışma ağacının güncel haline göre yapıldı. Önceki UX önerilerinden bazıları uygulanmış, bazıları kullanıcı tercihiyle geri alınmış durumda. Örneğin masa kartındaki bağlamsal `Sipariş Aç / Öde` CTA kaldırılmış; `Kaydet` ve `Mutfağa Gönder` dili de kullanıcı tercihiyle `Kaydet` olarak bırakılmıştır.

### 1. Mevcut frontend mimarisi özeti

Frontend React 18 + Vite tabanlı, ekran bazlı klasörlere ayrılmış bir POS uygulaması. Ana klasör yapısı:

- `client/src/components`: ekran ve UI bileşenleri.
- `client/src/context`: auth, toast, socket, incoming call gibi global contextler.
- `client/src/services`: tekil API client.
- `client/src/constants`: ortak enum/format yardımcıları.
- `client/src/utils`: küçük yardımcılar.
- `client/src/styles`: global CSS ve tasarım tokenları.

Ana route ve uygulama orkestrasyonu `client/src/App.jsx` içinde. Route componentleri `React.lazy` ile bölünmüş; bu önceki bundle riskine doğru bir cevap. Kanıt: `client/src/App.jsx:10-32`.

Mevcut yüksek seviyeli mimari:

- `AuthContext` kullanıcı/token state'ini ve role kontrolünü sağlar.
- `App.jsx` route ağacını, sidebar görünürlüğü, ödeme/hızlı ödeme modal state'ini ve sipariş ekranı wrapper'ını yönetir.
- Ekran bileşenleri kendi verilerini doğrudan `api` servisinden çeker.
- Socket context yalnız event subscribe mekanizması sağlar; domain eventlerinin nasıl yorumlanacağı ekranlara bırakılmıştır.

5. Backend kaynaklı veriler çoğunlukla ekran state'lerinde tutulur; merkezi query/cache katmanı yoktur.

Dosya boyutu ve sorumluluk yoğunluğu kanıtı:

| Dosya                                                | Satır | Değerlendirme                                                                                        |
|------------------------------------------------------|-------|------------------------------------------------------------------------------------------------------|
| client/src/components/orders/OrderScreen.jsx         | 1509  | P1 - Sipariş domaini, sepet, müşteri, transfer, ödeme gating, modal ve UI tek dosyada.               |
| client/src/components/tables/TablesScreen.jsx        | 1444  | P1 - Masa grid, transfer, paket yan panel, caller modal, action sheet ve socket/polling tek dosyada. |
| client/src/components/settings/PrinterDetailPage.jsx | 993   | P1 - Yazıcı formu, discovery, preview, encoding, print options ve kayıt akışı tek dosyada.           |
| client/src/components/settings/MenuSettingsPage.jsx  | 869   | P2 - Kategori, ürün, sıralama, bulk işlem ve modal state aynı dosyada.                               |
| client/src/components/reports/ReportsScreen.jsx      | 804   | P2 - Günlük rapor, kapalı siparişler, export, grafik ve filtre state'i aynı ekranda.                 |
| client/src/components/payments/SplitPaymentModal.jsx | 680   | P2 - Ayrı ödeme state makinesi modal içinde yoğun.                                                   |
| client/src/components/payments/PaymentScreen.jsx     | 497   | P2 - Ödeme state'i ve UI birlikte; bazı mantıklar hızlı ödeme ile tekrar ediyor.                     |
| client/src/services/api.js                           | 227   | P2 - Tüm domain endpointleri tek service/facade içinde.                                              |

Genel karar: Klasör ayrımı yüzeyde iyi, fakat gerçek mimari ekran dosyalarında toplanmış. Component sınırları "domain davranışı" üzerinden değil, çoğunlukla "ekran" üzerinden kurulmuş. Bu V1 için çalışır ama ürün büyüdükçe hızlı bozulur.

## 2. State akışı özeti

### 2.1 Global state kaynakları

| State kaynağı   | Dosya                                                          | Sorumluluk                                                                     | Risk                                                                                |
|-----------------|----------------------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Auth/user/token | <code>client/src/context/AuthContext.jsx:8-36</code>           | Kullanıcı, loading, login/logout, token set/reset.                             | Orta. Token <code>api</code> singleton ve <code>localStorage</code> ile çift bağlı. |
| Toast           | <code>client/src/context/ToastContext.jsx:7-17</code>          | Geçici bildirimler.                                                            | Düşük. Basit ve yeterli.                                                            |
| Socket          | <code>client/src/context/SocketContext.jsx:13-76</code>        | Socket bağlantısı ve event subscribe.                                          | Orta. Domain event yorumları ekranlara dağılmış.                                    |
| Incoming Call   | <code>client/src/context/IncomingCallContext.jsx:50-151</code> | Caller ID polling, banner, dismiss/open order.                                 | Orta-yüksek. UI, polling ve order navigation callback'i aynı contextte.             |
| API token       | <code>client/src/services/api.js:5-12</code>                   | Token <code>ApiService</code> instance içinde ve <code>localStorage</code> da. | Orta. AuthContext ile sıkı coupling.                                                |

Global state yeterince hafif tutulmuş; bu olumlu. Fakat veri fetching ve cache için bir katman yok. Her ekran kendi API çağrısını, loading/error state'ini ve refresh stratejisini kendi içinde tekrar kuruyor.

### 2.2 Route ve ekranlar arası veri akışı

Route yapısı `App.jsx` içinde hem erişim kontrolünü hem de ekran wiring'ini yönetiyor:

- Role bazlı erişim `ProtectedRoute` ile yapılıyor: `client/src/App.jsx:34-46`.
- Masa ekranından siparişe geçiş route state ile taşıyor: `client/src/App.jsx:73-75`.
- Paket sipariş state'i yine route state ile taşıyor: `client/src/App.jsx:77-88`.
- Ödeme ve hızlı ödeme modal state'i `App.jsx` içinde tutuluyor: `client/src/App.jsx:50-52`.
- Ödeme sonrası dönecek yer string ref ile belirleniyor: `client/src/App.jsx:94-124`.
- `OrderScreenWrapper`, route `location.state` içinden table/order/customer/callLog bilgilerini alıp props olarak geçiriyor: `client/src/App.jsx:260-286`.

Bu akış pratik ama kırılgan:

- `location.state` sayfa yenilemede kaybolabilir.
- Ödeme sonrası davranış `paymentAfterCompleteRef.current = 'back' | 'tables'` gibi string state ile yönetiliyor.
- `App.jsx` route config, payment modal orchestration, sidebar state ve display settings side effect'ini aynı yerde taşıyor.

### 2.3 Sipariş ekranı state akışı

`OrderScreen.jsx` aynı anda şu state kaynaklarını taşıyor:

- Menü verisi: `categories`, `products`, `activeCat`, `searchQuery`, `categoriesLoading` (`OrderScreen.jsx:26-36`).
- Lokal sepet: `cartItems` (`OrderScreen.jsx:30`).

- Backend siparişi: `existingOrder` (`OrderScreen.jsx:31`).
- Ürün detay/modifier modal state'i: `modifierModal`, `lineDetailModal` (`OrderScreen.jsx:32-34`).
- Transfer state'i: `moveModalOpen`, `emptyTables` (`OrderScreen.jsx:37-38`).
- Paket/müşteri state'i: `takeawayPhone`, `selectedCustomer`, `customerModalOpen`, `customerSearchQuery`, `customerList`, `newCustomer`, `selectedTakeawayAddress` (`OrderScreen.jsx:39-47`).
- Confirm modal state'i: `confirmDialog` (`OrderScreen.jsx:48`).

Ana state flow:

1. Kategori/ürünler ekran açılınca yüklenir: `OrderScreen.jsx:54`, `OrderScreen.jsx:107-123`.
2. Var olan sipariş `existingOrderId || table?.current_order_id` ile yüklenir: `OrderScreen.jsx:58-61`, `OrderScreen.jsx:126-129`.
3. Ürünler önce lokal `cartItems` içine eklenir: `OrderScreen.jsx:187-264`.
4. `handleSaveOrder`, mevcut sipariş varsa `api.addOrderItems`, yoksa `api.createOrder` çağırır: `OrderScreen.jsx:325-373`.
5. Toplamlar frontendde `savedTotal + cartSubtotal` olarak türetilir: `OrderScreen.jsx:603-607`.
6. Ödeme açma koşulu `existingOrder && !hasUnsavedChanges && existingOrder.status !== 'closed' && activeExistingItems.length > 0`: `OrderScreen.jsx:611-612`.

Bu model iş kuralını açıkça UI içine gömüyor. Özellikle `hasUnsavedChanges` ve `canOpenPayment` ödeme görünürlüğü için kritik policy, ama ekrana özel local expression olarak kalmış durumda.

## 2.4 Masa ekranı state akışı

`TablesScreen.jsx` yoğun bir state merkezi gibi davranıyor:

- Masa alanları ve aktif alan: `areas`, `activeArea`, `loading` (`TablesScreen.jsx:39-41`).
- Transfer modu: `transferMode` (`TablesScreen.jsx:42`).
- Paket sipariş queue: `takeawayOrders` (`TablesScreen.jsx:43`).
- Caller ID modal/history: `callModalOpen`, `callHistory`, `callHistoryLoading` (`TablesScreen.jsx:44-46`).
- Saat/timer: `now` (`TablesScreen.jsx:47`).
- Masa menü/confirm/cancel state'leri: `openMenuTableId`, `tableCancelTarget`, `tableCancelLoading`, `tableConfirm` (`TablesScreen.jsx:48-55`).
- Paket menü/confirm/loading state'leri: `openTakeawayMenuId`, `takeawayConfirmTarget`, `takeawayActionLoading` (`TablesScreen.jsx:51-53`).
- Transfer highlight state'i: `recentlyMovedTableId` (`TablesScreen.jsx:54`).

Ana side effect akışı:

- `loadTables` hem data yükler hem aktif area kararını verir: `TablesScreen.jsx:64-80`.
- `loadTakeaway` paket siparişleri ayrı endpointten çeker: `TablesScreen.jsx:83-91`.
- Route state değişimi, socket eventleri ve fallback polling aynı ekranı refresh eder: `TablesScreen.jsx:93-127`.
- `now` her saniye güncellenir: `TablesScreen.jsx:129-132`.
- ESC ve outside click davranışı local effectlerde yönetilir: `TablesScreen.jsx:140-161`.

Bu ekran masa operasyonu ile paket operasyonunu tek componentte birleştiriyor. Masa grid'i, paket yan paneli, caller ID modalı, transfer confirm'i ve action sheet aynı dosyada. Bu, frontend mimarisinin en kırılğan alanlarından

biri.

## 2.5 Ödeme/hızlı ödeme state akışı

Ödeme domaini üç ayrı componentte dağılmış:

- Ayrıntılı ödeme: `PaymentScreen.jsx`.
- Hızlı ödeme: `QuickPaymentModal.jsx`.
- Ayrı ödeme/split: `SplitPaymentModal.jsx`.

Tekrar eden hesaplama:

- `PaymentScreen` kendi `round2`, `paidTotal`, `totalDue`, `isFullyPaid` hesabını yapıyor: `PaymentScreen.jsx:20`, `PaymentScreen.jsx:61-67`.
- `QuickPaymentModal` aynı hesabı tekrar ediyor: `QuickPaymentModal.jsx:19`, `QuickPaymentModal.jsx:36-39`.
- `TablesScreen` masa kartı için benzer "tam ödendi mi" hesabı yapıyor: `TablesScreen.jsx:284-289`.
- `TablesScreen` paket ödeme durumu için ayrı helper yazmış: `TablesScreen.jsx:291-300`.
- `SplitPaymentModal` kendi payer/item allocation hesaplarını modal içinde tutuyor: `SplitPaymentModal.jsx:46-57`, `SplitPaymentModal.jsx:80-112`, `SplitPaymentModal.jsx:185-225`.

Bu tek başına bug değildir, ancak ödeme gibi kritik bir domain için risklidir. Tam ödeme toleransı `0.02`, kalan tutar, paid/open/closed ayrımı ve "masa kapatılabilir mi" gibi kurallar UI içinde farklı yerlerde tekrar edilirse, bir sonraki değişiklikte ekranlar ayrışır.

## 2.6 Ayarlar state akışı

Ayarlar route'ları bölünmüş; fakat her büyük ayar ekranı kendi küçük uygulaması gibi:

- `SettingsLayout` yalnız `Outlet` döndürüyor: `client/src/components/settings/SettingsLayout.jsx:1-4`. Ortak settings shell, breadcrumb, kaydedilmemiş değişiklik guard'ı, ortak form davranışı yok.
- `PrinterDetailPage` hem yazıcı formunu hem StoreBridge discovery'yi hem preview fetch'ini hem encoding seçeneklerini hem dirty tracking'i yönetiyor: `PrinterDetailPage.jsx:156-202`, `PrinterDetailPage.jsx:245-332`, `PrinterDetailPage.jsx:390-459`.
- Printer preview ayrı küçük component içinde debounced API call yapıyor: `PrinterDetailPage.jsx:65-106`. Bu iyi bir başlangıç, ama aynı dosyada kalmış.
- Kaydedilmemiş değişiklik guard'ı hala browser confirm ile çalışıyor: `PrinterDetailPage.jsx:386`.

Ayarlar tarafında ana sorun domain form modellerinin UI state'i olarak tutulması. `printOptions` gibi nested yapı çok sayıda `setPrintOptions` callback'ile güncelleniyor: `PrinterDetailPage.jsx:339-379`, `PrinterDetailPage.jsx:511-512`, `PrinterDetailPage.jsx:883-979`.

## 3. Kritik state sorunları

### P1 - Sipariş state'i tek ekranda aşırı parçalı ve çok kaynaklı

Kanıt:

- Lokal sepet `cartItems`, backend sipariş `existingOrder`, müşteri state'i `selectedCustomer`, paket adresi `selectedTakeawayAddress`, modal state'leri ve transfer state'i aynı component içinde: `OrderScreen.jsx:26-48`.

- Toplamlar frontendde kaydedilmiş sipariş + lokal sepet toplamı olarak türetiliyor: `OrderScreen.jsx:603-607`.
- Ödeme açma policy'si local expression: `OrderScreen.jsx:611-612`.

Etki:

- Kullanıcı ekranda bir toplam görürken backendde ödenebilir toplam farklı olabilir.
- Kaydet, ödeme, müşteri seçimi, transfer ve item edit gibi akışlar aynı `saving` bayrağına bağlanıyor; bu ileride yanlış disabled/loading davranışı üretir.
- Paket ve masa siparişleri aynı component state modelinde taşındığı için paket özel kurallar ekran boyunca koşullara dağılır.

Teknik kök neden:

- `OrderScreen` bir "container + view" ayrımına sahip değil.
- Sipariş draft state'i, order query state'i ve UI modal state'i tek componentte tutuluyor.
- Domain policy fonksiyonları ekran dışına çıkarılmamış.

Önerilen çözüm:

- Önce davranış değiştirmeden `useOrderDraft`, `useOrderCustomer`, `useOrderMutations`, `getOrderActionState` gibi küçük hook/utility sınırları çıkar.
- `cartItems` ve `existingOrder` için selector fonksiyonları yaz: `displayTotal`, `canOpenPayment`, `canEditItem`, `isClosed`, `hasUnsavedChanges`.
- Daha sonra `OrderTicketPanel`, `ProductGrid`, `OrderTopBar`, `CustomerPickerModal` alt componentleri oluştur.

## P1 - Masa ekranı data refresh kaynakları merkezi değil

Kanıt:

- İlk yükleme/route refresh: `TablesScreen.jsx:93-96`.
- Sidebar toggle/paket yükleme: `TablesScreen.jsx:98-104`.
- Socket event refresh: `TablesScreen.jsx:107-117`.
- Socket kopunca 30 saniye polling: `TablesScreen.jsx:120-127`.
- İşlem sonrası manuel refreshler: `TablesScreen.jsx:364-367`, `TablesScreen.jsx:406-408`, `TablesScreen.jsx:421-426`, `TablesScreen.jsx:460-464`.

Etki:

- Aynı verinin ne zaman tazeleneyeceği ekran boyunca dağılmış.
- Race condition riski var: socket event, polling ve işlem sonrası refresh aynı anda çalışabilir.
- Paket ve masa refreshleri birlikte çağrıldığı için gereksiz API trafiği oluşur.

Teknik kök neden:

- Query/cache katmanı yok.
- Masa ve paket sipariş state'i aynı ekran state'inde yönetiliyor.
- Event handling "hangi event hangi query'yi invalidate eder" yerine doğrudan `loadTables/loadTakeaway` çağrısı olarak yazılmış.

Önerilen çözüm:

- Düşük riskli ilk adım: `useTablesData({ includeTakeaway })` hook'u çıkar.
- Socket/polling invalidation mantığını hook içine taşı.
- Masa actions sonrası `refreshAll` tek fonksiyonla yürüsün; hangi endpointin yenileneceği hook içinde belirlensin.

## P1 - Ödeme kuralları üç componentte tekrar ediyor

Kanıt:

- `PaymentScreen`: `round2`, `paidTotal`, `totalDue`, `isFullyPaid`: `PaymentScreen.jsx:20`, `PaymentScreen.jsx:61-67`.
- `QuickPaymentModal`: aynı hesaplar: `QuickPaymentModal.jsx:19`, `QuickPaymentModal.jsx:36-39`.
- `TablesScreen`: masa paid hesabı: `TablesScreen.jsx:284-289`.
- `TablesScreen`: paket paid/state hesabı: `TablesScreen.jsx:291-300`.

Etki:

- Bir ödeme toleransı veya paid-state değişikliği bir ekranda güncellenip diğerinde unutulabilir.
- `paid_open`, `closed`, `unpaid`, `partial` gibi ürün durumları frontendde ekran bazlı hesaplandığı için UI tutarsızlığı riski artar.
- Ödeme/masa kapatma gibi para ve operasyon kuralı taşıyan alanlarda bu tekrar kabul edilebilir değil.

Teknik kök neden:

- `paymentPolicy` veya `orderStatusSelectors` gibi merkezi saf yardımcılar yok.
- UI componentleri domain state derive ediyor.

Önerilen çözüm:

- `client/src/utlis/orderPaymentState.js` gibi saf utility oluştur.
- `getPaymentSummary(order)`, `isOrderFullyPaid(order)`, `canCloseOrder(order)`, `getTakeawayPaymentState(order)` fonksiyonları tek kaynak olsun.
- Önce mevcut hesapları birebir taşı; davranış değişikliği yapma.

## P2 - Route state uygulama akışında kalıcı kaynak gibi kullanılıyor

Kanıt:

- Masa siparişine geçişte table ve existingOrderId route state ile veriliyor: `App.jsx:73-75`.
- Paket siparişte customer/existingOrderId/prefillPhone/callLogId route state ile veriliyor: `App.jsx:77-88`.
- `OrderScreenWrapper` bu state'i doğrudan props'a çeviriyor: `App.jsx:260-286`.

Etki:

- Sayfa yenileme veya deep link ile açmada UI bağlamı eksilebilir.
- Masa adı/salon alanı gibi gösterim bilgileri bazen state ile gelir, bazen backend sipariştten türetilir.
- Caller ID'den açılan sipariş ile normal paket sipariş aynı route üzerinde state farkıyla ayırır; bu debugging'i zorlaştırır.

Teknik kök neden:

- URL parametreleri, query parametreleri ve server kaynaklı hydrate modeli net ayrılmamış.

Önerilen çözüm:

- `orderType`, `existingOrderId`, `callLogId` gibi kalıcı akış parametrelerini URL/query ile taşımayı değerlendir.
- `OrderScreen` route state yoksa ilgili table/order bilgisini endpointten hydrate edebilmeli.
- İlk düşük riskli adım: wrapper içinde route-state normalize eden `buildOrderScreenProps(location.state, params)` utility yaz.

## P2 - Async hata yönetimi ekran bazlı ve standart değil

Kanıt:

- `OrderScreen` birçok yerde `toast.error(err.message)` ve bazı yerde fallback mesaj kullanıyor: `OrderScreen.jsx:107-123`, `OrderScreen.jsx:325-380`, `OrderScreen.jsx:547-584`.
- `TablesScreen` işlem sonrası refresh ve hata yönetimini her handler içinde tekrar ediyor: `TablesScreen.jsx:401-445`.
- `ApiService` non-JSON yanıt ve 401 davranışını global yönetiyor, fakat request cancellation veya retry yok: `api.js:15-59`.
- `PrinterPreviewPanel` kendi debounce/fetch error state'ini local tutuyor: `PrinterDetailPage.jsx:84-106`.

Etki:

- Aynı API hatası farklı ekranlarda farklı kullanıcı metniyle görünür.
- Unmount sırasında devam eden requestler için sistematik abort yok.
- Hızlı tıklama / yavaş network koşullarında stale response riski artar.

Teknik kök neden:

- Ortak mutation/query helper yok.
- Loading state'leri domain operasyonuna özel değil, genellikle tek `saving` veya `processing` bayrağı.

Önerilen çözüm:

- Düşük riskli: `getErrorMessage(err, fallback)` helper ve `withToastError` wrapper.
- Orta riskli: küçük custom hooklar (`useAsyncAction`, `useDebounceQuery`) ile loading/error standardı.
- Uzun vadeli: React Query veya benzeri query/cache invalidation katmanı.

## 4. Component mimarisi sorunları

### P1 - `OrderScreen.jsx` tek sorumluluğu açıkça aşıyor

Kanıt:

- 1509 satır.
- API orchestration: `OrderScreen.jsx:107-123`, `OrderScreen.jsx:325-373`, `OrderScreen.jsx:547-599`.
- Business rules: `OrderScreen.jsx:211-223`, `OrderScreen.jsx:611-612`, `OrderScreen.jsx:968`, `OrderScreen.jsx:1041-1046`.
- UI rendering: `OrderScreen.jsx:642-1245` ve modal renderları `OrderScreen.jsx:1255-1415`.
- Aynı dosyada nested `ModifierModal` ve `ClipboardEmpty` componentleri var: `OrderScreen.jsx:1505-1575`.

Etki:



- Sipariş ekranında yapılacak küçük UI değişikliği bile ödeme gating veya item lifecycle davranışına temas edebilir.
- Test yazmak pahalıdır; saf fonksiyon sınırları azdır.
- Yeni geliştirici için dosyayı anlamak operasyon akışını ezberlemeye bağlıdır.

Önerilen ayrıştırma sırası:

1. Saf selector/policy fonksiyonları.
2. API mutation hookları.
3. UI alt componentleri.
4. Modal componentlerini bağımsızlaştırma.

## **P1 - TablesScreen.jsx iki ayrı operasyon ekranını tek componentte taşıyor**

Kanıt:

- Masa grid state'i ve paket sipariş queue state'i aynı componentte: `TablesScreen.jsx:39-55`.
- Masa transfer/ödeme/kapatma/yazdırma action handlerları: `TablesScreen.jsx:207-399`.
- Paket teslimat/iptal/yazdırma handlerları: `TablesScreen.jsx:401-468`.
- Masa kartları ve paket sipariş yan paneli aynı render gövdesinde: `TablesScreen.jsx:599-1115`.
- Confirm/action modal renderları aynı dosyanın sonunda: `TablesScreen.jsx:1159-1425`.

Etki:

- Salon ve paket operasyonunun değişiklik ritimleri farklıdır; tek dosyada değıştikçe regresyon alanı büyür.
- Masa event refreshleri paket refreshleriyle gereksiz birleşir.
- Action sheet, confirm modal ve paket menu gibi reusable parçalar kopyalanmaya aday kalır.

Önerilen ayrıştırma:

- `useTablesData`, `useTakeawayQueue`, `useTableActions`.
- `TableCard`, `TableActionSheet`, `TakeawaySidebar`, `TakeawayOrderCard`, `ConfirmDialog`.
- Paket sipariş için V1'de yan panel kalabilir; component sınırı yine ayrılmalı.

## **P1 - PrinterDetailPage.jsx form state ve entegrasyon state'ini tek yerde biriktiriyor**

Kanıt:

- Form state listesi: `PrinterDetailPage.jsx:156-181`.
- Dirty snapshot hesabı: `PrinterDetailPage.jsx:185-202`.
- Printer settings/discovery load: `PrinterDetailPage.jsx:245-332`.
- Save payload normalizasyonu: `PrinterDetailPage.jsx:390-459`.
- Encoding/advanced alanları: `PrinterDetailPage.jsx:676-741`.
- Layout/output/copy/role option alanları: `PrinterDetailPage.jsx:883-979`.

Etki:

- Yazıcı gibi sahada en kırılgan entegrasyonlardan biri UI dosyası içinde fazla karmaşık.
- Encoding, physical printer binding ve print layout optionları aynı nested `printOptions` state'inde dağılıyor.
- Yeni printer türü veya profil eklendiğinde dosya daha da büyür.

Önerilen ayırıştırma:

- `usePrinterFormModel` .
- `PrinterConnectionSection` .
- `PrinterAdvancedEncodingSection` .
- `PrinterLayoutOptionsSection` .
- `PrinterPreviewPanel` ayrı dosyaya taşınmalı.

## P2 - API client domain sınırı taşımıyor

Kanıt:

- Auth, tables, products, orders, payments, customers, caller ID, reports, reservations, stock, waiter call, print ve admin endpointleri tek class içinde: `api.js:73-227` .
- Token state aynı class içinde ve `localStorage` ile yönetiliyor: `api.js:5-12` .
- 401 durumunda doğrudan `window.location.reload()` yapılıyor: `api.js:31-34` .

Etki:

- Domain bazlı test/mocking zor.
- UI ekranları tüm API yüzeyine erişebilir; yanlış domain çağrısı için sınır yok.
- Auth failure davranışı UI routing yerine sayfa reload ile çözülüyor; Electron içinde kaba bir recovery modeli.

Önerilen çözüm:

- Davranış korumalı olarak `ordersApi` , `tablesApi` , `paymentsApi` , `adminPrintersApi` gibi modüllere ayır.
- Mevcut default `api` facade bir süre korunabilir.
- Auth reset için callback/event mekanizması eklenebilir; reload son seçenek olmalı.

## P2 - Settings shell yok, ayarlar ekranları ortak davranış paylaşmıyor

Kanıt:

- `SettingsLayout` yalnız `Outlet` döndürüyor: `SettingsLayout.jsx:1-4` .
- Printer detail kendi dirty guard'ını local `window.confirm` ile yapıyor: `PrinterDetailPage.jsx:386` .
- Menu settings kendi delete/bulk confirm'lerini browser confirm ile yapıyor: `MenuSettingsPage.jsx:372` , `MenuSettingsPage.jsx:391` , `MenuSettingsPage.jsx:505` .

Etki:

- Ayarlar ekranları farklı kaydet/çık/confirm davranışları geliştirir.
- Profesyonel admin deneyimi yerine birbirinden bağımsız formlar hissi oluşur.

Önerilen çözüm:

- `SettingsPageShell` , `SettingsSection` , `UnsavedChangesGuard` , `ConfirmDialog` gibi ortak yapılar çıkar.
- İlk turda sadece ortak confirm bileşeni ile başlayabilir.

## 5. Amatör görünen frontend alanları

### P1 - Business rule'ların UI içine gömülmesi

Örnekler:

- Siparişte ödeme açma koşulu: `OrderScreen.jsx:611-612`.
- Existing item edit kuralı: `OrderScreen.jsx:968`.
- Masa paid kuralı: `TablesScreen.jsx:284-289`.
- Paket ödeme state'i: `TablesScreen.jsx:291-300`.
- Ödeme kapatma kuralı: `PaymentScreen.jsx:143-146`.

Neden amatör görünüyor:

Bu kurallar domain policy olarak tek kaynakta olmalı. UI içinde kalınca her ekran kendi doğruluğunu üretir; ürün davranışı "component hangi koşulu yazdıysa" ona dönüşür.

### P1 - Monolitik ekran bileşenleri

Örnekler:

- `OrderScreen.jsx` : 1509 satır.
- `TablesScreen.jsx` : 1444 satır.
- `PrinterDetailPage.jsx` : 993 satır.

Neden amatör görünüyor:

Ekran dosyaları hem state makinesi hem API orchestration hem UI template hem modal registry gibi çalışıyor. Bu yapı ürün olgunlaştıkça "her şeyi bilen component" anti-pattern'ine döner.

### P1 - Tekrar eden ödeme hesapları

Örnekler:

- `round2`, `paidTotal`, `totalDue`, `isFullyPaid` tekrarları: `PaymentScreen.jsx:20`, `PaymentScreen.jsx:61-67`, `QuickPaymentModal.jsx:19`, `QuickPaymentModal.jsx:36-39`.
- Masa paid hesabı ayrı: `TablesScreen.jsx:284-289`.

Neden amatör görünüyor:

Ödeme domaininde tolerans ve tamamlanma kuralı UI componentlerinde tekrar edilmemeli. Bu para/kasa davranışıdır.

### P2 - Browser confirm kullanımı hala ayarlar/menü tarafında duruyor

Örnekler:

- Printer çıkış guard'ı: `PrinterDetailPage.jsx:386`.
- Menü kategori/ürün/bulk confirm: `MenuSettingsPage.jsx:372`, `MenuSettingsPage.jsx:391`, `MenuSettingsPage.jsx:505`.

Neden amatör görünüyor:

Önceki turda masa/sipariş confirmleri iyileştirilmiş olsa da ayarlar tarafında native confirm kalmış. Aynı ürün içinde bazı kritik işlemler custom modal, bazıları browser dialog ile ilerliyor.

## P2 - Inline style yoğunluğu component sistemini zayıflatıyor

Kanıt:

- Masa kartı, action sheet, paket kartı stilleri büyük oranda inline: `TablesScreen.jsx:599-1115`.
- Sipariş ekranı ürün grid/adisyon/topbar stilleri inline: `OrderScreen.jsx:642-1245`.
- Payment ekranı kart ve aksiyon layoutları inline: `PaymentScreen.jsx:190-491`.
- Split payment modal kendi `<style>` bloğunu taşıyor: `SplitPaymentModal.jsx:487-707`.

Neden amatör görünüyor:

Tasarım sistemi tokenları var, ama uygulama yüzeyi ad hoc inline stillerle büyüyor. Bu tekrarı artırır ve UI kalite standardını merkezi yönetmeyi zorlaştırır.

## P2 - Çok fazla ekran kendi mini data layer'ını yazıyor

Örnekler:

- Customers import/export/load/stats state'i tek componentte: `CustomersScreen.jsx:8-26`, `CustomersScreen.jsx:29-235`.
- Reports günlük rapor, closed order pagination, export ve analytics state'i tek componentte: `ReportsScreen.jsx:247-400`.
- Menu settings kategori/ürün/bulk/sıralama state'i tek componentte: `MenuSettingsPage.jsx:246-263`.

Neden amatör görünüyor:

Ekranlar domain hook'ları yerine local state ve API handler yığınıyla büyüyor. Bu pattern yeni ekranlarda kopyalanır.

## 6. Teknik borç ve kırılabilirlik alanları

### P1 - Merkezi frontend domain policy yok

Alanlar:

- Sipariş aksiyon görünürlüğü.
- Masa aksiyon görünürlüğü.
- Ödeme tamamlanma/tam kapatma toleransı.
- Paket teslimat/ödeme ilişkisi.
- Item edit/void yetkisi.

Sonuç:

Backend guard'ları olsa bile frontend davranışı ekran bazında değişir. POS gibi hızlı kullanımda kullanıcı farklı ekranlarda farklı kural görür.

### P1 - Ekran state'i ile backend state'i arasında cache/invalidasyon modeli yok

Alanlar:

- Tables socket/polling/route refresh karışımı.
- Order save sonrası bazen full order reload, bazen result set etme: `OrderScreen.jsx:363-370`.
- Payment completion sonrası navigation ile refresh tetikleme: `App.jsx:117-124`.
- Split payment sonrası state reload: `SplitPaymentModal.jsx:221-225`.

Sonuç:

Stale UI riski düşük/orta düzeyde var. Özellikle hızlı ödeme, masa kapatma, socket event ve polling aynı anda çalıştığında görülebilir.

## P2 - Prop drilling ve callback orchestration büyümeye açık

Kanıt:

- `TablesScreen` props ile `onOpenOrder`, `onPayment`, `onQuickPayment`, `onOpenTakeawayOrder` alıyor: `TablesScreen.jsx:33-37`.
- `OrderScreenWrapper` route state'i props'a çevirip `OrderScreen` e geçiriyor: `App.jsx:260-286`.
- `OrderScreen` ödeme ve navigation callbacklerini props ile alıyor: `OrderScreen.jsx:13-24`.

Sonuç:

Şimdilik yönetilebilir. Ancak yeni ödeme aksiyonları, caller ID, paket queue ve masa transfer dönüşleri arttıkça callback zinciri belirsizleşir.

## P2 - Frontend test/lint/typecheck kapısı yok

Kanıt:

- `client/package.json:6-9` yalnız `dev`, `build`, `preview` scriptlerini içeriyor.

Sonuç:

Frontend mimari refactorları için güvenlik ağı yok. Bu yüzden her parçalama önce saf utility extraction ve build doğrulamasıyla küçük adımlara bölünmeli.

## P2 - Settings formları nested state'i doğrudan güncelliyor

Kanıt:

- `printOptions` birçok yerde local callback ile güncelleniyor: `PrinterDetailPage.jsx:339-379`, `PrinterDetailPage.jsx:511-512`, `PrinterDetailPage.jsx:883-979`.
- Dirty snapshot JSON stringify ile kontrol ediliyor: `PrinterDetailPage.jsx:185-202`.

Sonuç:

Nested state shape değişirse form dirty logic, save payload ve preview aynı anda etkilenir. Yazıcı ayarları gibi cihaz bağımlı alanda bu risk önemlidir.

# 7. Hemen ele alınması gerekenler

## P1 - OrderScreen için domain selector/policy extraction

Kapsam:

- `hasUnsavedChanges`, `canOpenPayment`, `displayTotal`, `canEditQty`, `isLineReadOnly`.

- Davranış deęiřtirmeden saf fonksiyonlara çıkarılmalı.

Neden hemen:

- En büyük frontend dosyası.
- Sipariş ve ödeme geçiři en kritik POS akışı.
- Düşük riskli ilk refactor ile test edilebilir sınır oluşur.

## P1 - Ödeme durum hesaplarını tek utility'ye toplama

Kapsam:

- `round2`
- `getPaidTotal(order)`
- `getOrderTotal(order)`
- `getTotalDue(order)`
- `isFullyPaid(order)`
- `canCloseOrder(order)`
- `getTakeawayPaymentState(order)`

Neden hemen:

- Aynı kural `PaymentScreen`, `QuickPaymentModal`, `TablesScreen` içinde tekrar ediyor.
- Para/kasa davranışı merkezi olmalı.

## P1 - TablesScreen data/action hook ayrımı

Kapsam:

- `useTablesData`
- `useTakeawayQueue`
- `useTableActions`
- `TableActionSheet` ve `TakeawaySidebar` component extraction.

Neden hemen:

- Masa ve paket operasyonu tek dosyada aşırı büyümüş.
- Refresh kaynakları dağılmış.

## P2 - Ortak ConfirmDialog bileřeni

Kapsam:

- Masa/sipariş tarafında yeni custom confirmler var.
- Ayarlar/menü tarafında hala `window.confirm` var.

Neden:

- Düşük riskli.
- UI tutarlılığını artırır.
- Sonraki refactorlar için modal standardı oluşturur.

## P2 - Settings shell ve printer form model ayrımı

Kapsam:

- `SettingsLayout` gerçek shell'e dönüşmeli.
- `PrinterDetailPage` form model/hook ve section componentlerine bölünmeli.

Neden:

- Yazıcı ayarı sahada kritik.
- Şu an dosya hem cihaz integration hem UI hem form normalize işlemlerini taşıyor.

## 8. Sonraki turda uygulanabilecek düşük riskli refactor önerileri

Bu turda uygulanmadı; aşağıdaki liste davranış değiştirmeden küçük PR/tur olarak ele alınabilir.

### 8.1 `orderPaymentState` utility

Önerilen dosya:

- `client/src/utls/orderPaymentState.js`

İçerik:

- `roundMoney(value)`
- `getPaidTotal(order)`
- `getOrderTotal(order)`
- `getTotalDue(order)`
- `isOrderFullyPaid(order)`
- `canCloseOrder(order)`
- `getPaymentStateLabel(order)`

Etkilenecek ekranlar:

- `PaymentScreen.jsx`
- `QuickPaymentModal.jsx`
- `TablesScreen.jsx`
- ileride `SplitPaymentModal.jsx`

Risk:

- Düşük/orta. İlk adımda birebir mevcut hesaplar taşınmalı, davranış değişmemeli.

### 8.2 `orderActionPolicy` utility

Önerilen dosya:

- `client/src/utls/orderActionPolicy.js`

İçerik:

- `canOpenPayment({ existingOrder, cartItems })`
- `canEditOrderItem(item, order)`
- `canVoidOrderItem(item, order, user)`

- `canSaveDraft({ cartItems, orderType, selectedCustomer })`

Etkilenecek ekranlar:

- `OrderScreen.jsx`
- ileride masa action sheet ve ödeme yönlendirmeleri.

Risk:

- Düşük. Önce unit test yazılmalı.

### 8.3 `OrderScreen` için küçük hook'lar

Tek büyük hook önerilmez. Aşamalı:

- `useOrderCatalog` : kategori, ürün, arama.
- `useOrderDraftCart` : lokal sepet işlemleri.
- `useOrderMutations` : save, item update, void.
- `useOrderCustomer` : müşteri seçimi/paket adresi.

Risk:

- Orta. UI ile state sıkı bağlı olduğu için tek turda değil, küçük parçalarda yapılmalı.

### 8.4 `TablesScreen` parçalama

Önerilen componentler:

- `TableCard`
- `TableActionSheet`
- `TakeawaySidebar`
- `TakeawayOrderCard`
- `TableConfirmDialog`

Önerilen hooklar:

- `useTablesData`
- `useTakeawayOrders`
- `useTableTransfer`

Risk:

- Orta. Render çıktısı çok inline style içerdiği için önce component extraction, sonra CSS cleanup yapılmalı.

### 8.5 Ortak async action helper

Önerilen hook:

- `useAsyncAction({ onError })`

Amaç:

- `saving`, `processing`, `loading` bayraklarını operasyon bazlı standardize etmek.
- Hata mesajlarını tek helper ile normalize etmek.

Risk:



- Düşük. Önce yeni kodda kullanılmalı, sonra eski handlerlar taşınmalı.

## 8.6 Ortak `ConfirmDialog`

Önerilen component:

- `client/src/components/common/ConfirmDialog.jsx`

Kullanım hedefleri:

- `OrderScreen` `confirmDialog`.
- `TablesScreen` `tableConfirm/tableCancel/takeawayCancel`.
- `PrinterDetailPage` `unsaved changes`.
- `MenuSettingsPage` `delete/bulk confirm`.

Risk:

- Düşük. Önce sadece yeni modal API'si oluşturulup bir ekranda uygulanmalı.

## 8.7 Settings shell

Önerilen componentler:

- `SettingsPageShell`
- `SettingsToolbar`
- `SettingsUnsavedChangesGuard`

Risk:

- Düşük/orta. Görsel davranış etkileyebilir, bu yüzden önce yalnız layout wrapper olarak başlanmalı.

## Önceliklendirilmiş frontend refactor sırası

### Hemen

1. P1 - Ödeme state selector utility'si.
2. P1 - Sipariş aksiyon policy utility'si.
3. P1 - `OrderScreen` içinden saf hesap/koşul fonksiyonlarının çıkarılması.
4. P2 - Ortak `ConfirmDialog` standardı.

### Bu hafta

1. P1 - `TablesScreen` data hook ve `TakeawaySidebar` extraction.
2. P1 - `OrderScreen` katalog/cart/customer hooklarına ayrıştırma başlangıcı.
3. P2 - `PrinterDetailPage` form modelini hook'a alma.
4. P2 - API client domain modüllerine davranış korumalı facade ile bölme.

### Sonra

1. React Query veya hafif query/cache invalidation katmanı.
2. Route state bağımlılığını URL/server hydrate modeline taşıma.
3. Split payment state machine'i ayrı reducer/hook yapısına taşıma.

4. Frontend test altyapısı: utility unit testleri, sonra Playwright smoke.
5. Inline style borcunu component CSS/tasarım sistemi katmanına taşıma.

## Son karar

Frontend çalışır durumda ve route lazy loading gibi doğru yönde adımlar var. Ancak profesyonel ürün kalitesi açısından frontend mimarisi halen "ekran dosyaları içinde büyüyen local app" seviyesinde. En büyük risk, business rule'ların ekran componentlerine gömülmesi ve ödeme/sipariş/masa state'inin merkezi policy olmadan tekrar hesaplanması.

Bu noktada büyük bir mimari dönüşüm yapmak doğru değil. Doğru hareket, önce davranışı değiştirmeyen saf utility ve hook extraction ile test edilebilir sınırlar oluşturmak; sonra büyük ekranları parça parça küçültmek.